

MACHINE LEARNING FUNDAMENTALS SERIES

Multi-Output Weight Updates & Vectorized Backpropagation

Multivariable Chain Rule, GPU Matrix Computations, and Simultaneous Weight Optimization

1. Introduction and Benchmark Architecture

In deep neural networks with multiple output nodes (such as multi-class classification using **Softmax**), hidden layer weights and biases influence **multiple output predictions simultaneously**. To illustrate how GPUs process these connections at scale, we use a benchmark architecture:

- **Input Layer:** 2 input features (x_1, x_2).
- **Hidden Layer:** 1 hidden layer with 3 nodes using **ReLU** activation ($\text{ReLU}(z) = \max(0, z)$).
- **Output Layer:** 3 output logits (z_3, z_4, z_5) converted into probabilities via **Softmax**.

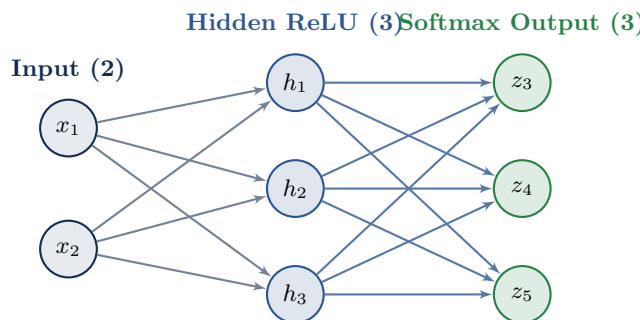


Figure 1: Benchmark Neural Network Topology: 2 Inputs $\rightarrow$ 3 Hidden ReLU Nodes $\rightarrow$ 3 Softmax Outputs.

2. Key Principles of Multi-Output Weight Updates

2.1 Simultaneous Accumulation over Iterations

Weights in a neural network are **not** updated step-by-step for each individual output. All gradients contributed by every output path are accumulated first across the sample or batch, and all parameters are updated **once simultaneously**.

2.2 Multivariable Chain Rule for Shared Parameters

When a hidden parameter (such as bias b_1 or weight w connected to h_1) fans out to affect all three output logits (z_3, z_4, z_5), its total gradient relative to Cross-Entropy Loss (L) is the sum of partial derivatives along all outgoing paths:

$$\frac{\partial L}{\partial b_1} = \sum_{i=3}^5 \frac{\partial L}{\partial z_i} \cdot \frac{\partial z_i}{\partial b_1} = \underbrace{\frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial b_1}}_{\text{Path from Output 3}} + \underbrace{\frac{\partial L}{\partial z_4} \cdot \frac{\partial z_4}{\partial b_1}}_{\text{Path from Output 4}} + \underbrace{\frac{\partial L}{\partial z_5} \cdot \frac{\partial z_5}{\partial b_1}}_{\text{Path from Output 5}} \quad (1)$$

Where: $\frac{\partial L}{\partial z_i} = (q_i - y_i)$ is prediction error at output node i (q_i is predicted Softmax probability, y_i is true target), and $\frac{\partial z_i}{\partial b_1}$ measures influence on logit z_i .

3. Vectorized Matrix Formulations (GPU Computation)

To process data efficiently on GPUs without slow Python loops, frameworks like PyTorch and TensorFlow stack individual vectors into matrices for batch processing.

3.1 Variables and Tensor Dimensions

Assuming a batch size of m (e.g., $m = 3$ samples):

Symbol	Description	Formula / Source	Tensor Shape
X	Input Data Matrix	Stacked sample features	$(m \times 2)$
$W^{(1)}$	Layer 1 Weights (Input $\rightarrow$ Hidden)	Feature weights	(2×3)
$b^{(1)}$	Layer 1 Biases (Hidden Nodes)	Node offsets	(1×3)
H	Hidden Activations	$\text{ReLU}(XW^{(1)} + b^{(1)})$	$(m \times 3)$
$W^{(2)}$	Layer 2 Weights (Hidden $\rightarrow$ Output)	Output weights	(3×3)
$b^{(2)}$	Layer 2 Biases (Output Nodes)	Logit offsets	(1×3)
Q	Softmax Predictions	$\text{Softmax}(HW^{(2)} + b^{(2)})$	$(m \times 3)$
Y	True One-hot Target Labels	Target distribution	$(m \times 3)$

Table 1: Dimension Specification for 2-3-3 Vectorized Architecture (m samples).

4. Step-by-Step Vectorized Backpropagation

4.1 Step 1: Output Layer Error Matrix ($dZ^{(2)}$)

The error matrix representing $(q - y)$ for all m samples across all 3 outputs is computed in a single element-wise matrix subtraction:

$$dZ^{(2)} = Q - Y \quad [\text{Shape: } m \times 3] \quad (2)$$

4.2 Step 2: Gradients for Output Layer Parameters ($dW^{(2)}, db^{(2)}$)

Multiplying transpose of hidden activation matrix H^T by output error matrix $dZ^{(2)}$ automatically sums error contributions across all m samples:

$$dW^{(2)} = \frac{1}{m} \cdot H^T \cdot dZ^{(2)} \quad [\text{Shape: } 3 \times 3], \quad db^{(2)} = \frac{1}{m} \cdot \text{sum_rows}(dZ^{(2)}) \quad [\text{Shape: } 1 \times 3] \quad (3)$$

4.3 Step 3: Error Backpropagation to Hidden Layer ($dZ^{(1)}$)

To pass prediction errors backward through the hidden layer, the computer multiplies output error matrix by transpose of output weight matrix $(W^{(2)})^T$, multiplied element-wise ($\odot$) by ReLU derivative $\sigma'(Z^{(1)})$ ($\sigma' = 1$ if $Z^{(1)} > 0$ else 0):

$$dZ^{(1)} = (dZ^{(2)} \cdot (W^{(2)})^T) \odot \sigma'(Z^{(1)}) \quad [\text{Shape: } m \times 3] \quad (4)$$

Mathematical Mechanism of Vectorization

The matrix product $dZ^{(2)} \cdot (W^{(2)})^T$ is the exact linear algebra mechanism that collects, routes, and sums error paths from all 3 output nodes into the 3 hidden nodes simultaneously—eliminating loops entirely.

4.4 Step 4: Gradients for Input Layer Parameters ($dW^{(1)}, db^{(1)}$)

Finally, gradients for the innermost weights and biases are calculated using input feature matrix X :

$$dW^{(1)} = \frac{1}{m} \cdot X^T \cdot dZ^{(1)} \quad [\text{Shape: } 2 \times 3], \quad db^{(1)} = \frac{1}{m} \cdot \text{sum_rows}(dZ^{(1)}) \quad [\text{Shape: } 1 \times 3] \quad (5)$$

5. Simultaneous Parameter Update (Gradient Descent)

Once gradient matrices $dW^{(1)}, db^{(1)}, dW^{(2)}, db^{(2)}$ are fully populated with accumulated batch gradients, the optimization algorithm (SGD with learning rate η) performs **one single tensor subtraction per layer**:

$$W^{(2)} \leftarrow W^{(2)} - \eta \cdot dW^{(2)}, \quad b^{(2)} \leftarrow b^{(2)} - \eta \cdot db^{(2)} \quad (6)$$

$$W^{(1)} \leftarrow W^{(1)} - \eta \cdot dW^{(1)}, \quad b^{(1)} \leftarrow b^{(1)} - \eta \cdot db^{(1)} \quad (7)$$

The Grand Finale: Parallel Execution

Every single weight and bias in the entire network changes its state at this exact instant—all at once in GPU memory. Vectorization transforms hundreds of scalar loops into 4 fast, parallel matrix operations.